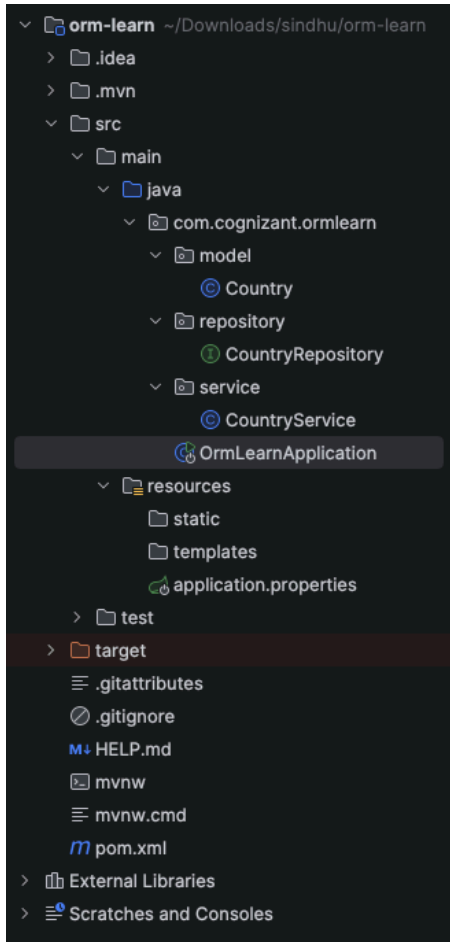


SPRING-DATA-JPA-HANDSON-1

Quick Example (Hands-on 1)



```
[mysql> SHOW TABLES;
+-----+
| Tables_in_db |
+-----+
| country      |
+-----+
1 row in set (0.15 sec)

[mysql> SELECT * FROM country;
+-----+-----+
| co_code | co_name |
+-----+-----+
| IN      | India   |
| JP      | Japan   |
| US      | United States |
+-----+-----+
3 rows in set (0.00 sec)
```

```

spring.application.name=orm-learn
server.port=8082

# Database configuration

spring.datasource.url=jdbc:mysql://localhost:3306/db
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.properties.hibernate.format_sql=true

```

```

package com.cognizant.ormlearn.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="country")
public class Country {
    @Id
    @Column(name = "co_code")
    private String code;

    @Column(name = "co_name") 4 usages
    private String name;
    public Country() {
    }
    public Country(String code, String name) { no usages
        this.code = code;
        this.name = name;
    }
    public String getCode() {
        return code;
    }
    public void setCode(String code) {
        this.code = code;
    }

    public String getName() { no usages
        return name;
    }
    public void setName(String name) { no usages
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country{" +
            "code='" + code + '\'' +
            ", name='" + name + '\'' + '}';
    }
}

```

```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository no usages
public interface CountryRepository extends JpaRepository<Country,String>{

}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service no usages
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional no usages
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }

}

```

HANDSON 4-

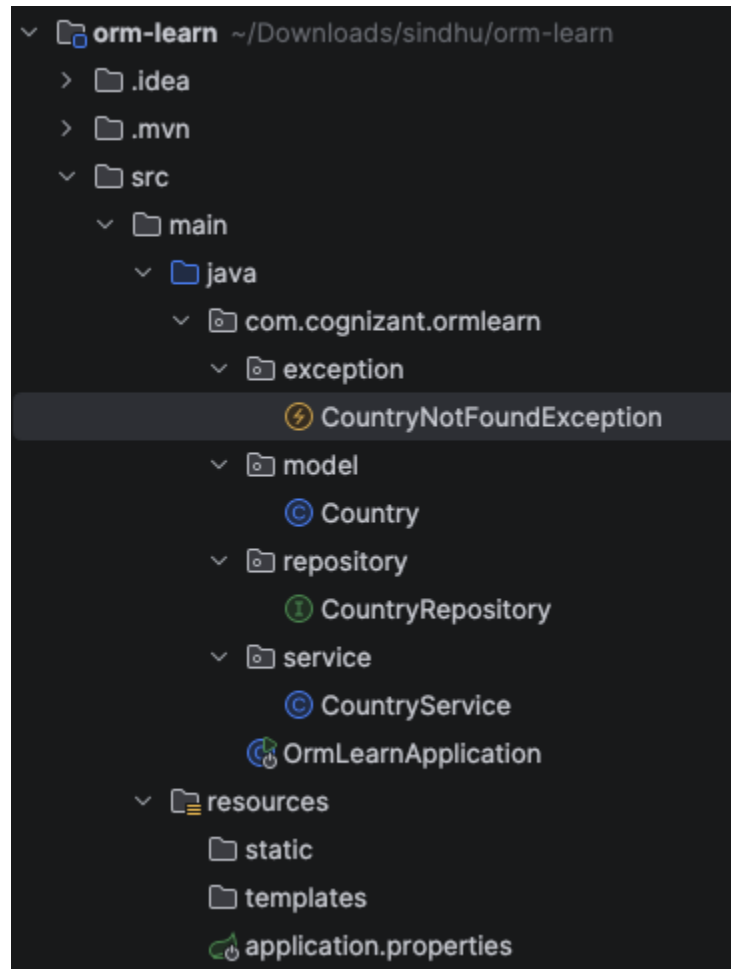
Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Abstraction over JPA
Implementation	No	Yes	Uses Hibernate internally
Purpose	Defines persistence API	Maps Java objects to database tables	Reduces boilerplate CRUD code
CRUD Support	Through implementation	Yes	Yes (using <code>JpaRepository</code>)
SQL Writing	Less	Less	Almost none
Transaction Management	No	Partial	Yes (<code>@Transactional</code>)

HANDSON 5: Find Country by Code

```

baseConfiguration$spawnWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore
.boot.tomcat.TomcatWebServer           : Tomcat started on port 8082 (http) with context path '/'
.ormlearn.OrmLearnApplication           : Started OrmLearnApplication in 2.639 seconds (process r
.ormlearn.OrmLearnApplication           : Start
.ormlearn.OrmLearnApplication           : Country = Country{code='IN', name='India'}
.ormlearn.OrmLearnApplication           : End

```



```

package com.cognizant.ormlearn.exception;

public class CountryNotFoundException extends Exception { 6 usages

    public CountryNotFoundException(String message) { 1 usage
        super(message);
    }
}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.exception.CountryNotFoundException;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Service 3 usages
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional 1 usage
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }

    @Transactional 1 usage
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {

        Optional<Country> result =
            countryRepository.findById(countryCode);

        if(result.isEmpty()) {
            throw new CountryNotFoundException("Country Not Found");
        }

        return result.get();
    }
}

```

```

package com.cognizant.ormlearn;

import ...

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService; 3 usages

    public static void main(String[] args)
        throws CountryNotFoundException {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        testFindCountryByCode();
    }

    private static void testGetAllCountries(){ no usages
        LOGGER.info("Start");
        List<Country> countries=countryService.getAllCountries();
        LOGGER.info("Countries: {}",countries);
        LOGGER.info("End");
    }

    private static void testFindCountryByCode() 1 usage
        throws CountryNotFoundException {

        LOGGER.info("Start");

        Country country =
            countryService.findCountryByCode( countryCode: "IN");

        LOGGER.info("Country = {}", country);

        LOGGER.info("End");
    }
}

```

Hands-on 7: Add Country

```
[mysql> SELECT * FROM country;
```

co_code	co_name
IN	India
JP	Japan
US	United States

```
3 rows in set (0.00 sec)
```

```
[mysql> SELECT * FROM country;
```

co_code	co_name
AU	Australia
IN	India
JP	Japan
US	United States

```
4 rows in set (0.02 sec)
```

```
@Transactional no usages
public void addCountry(Country country) {
    countryRepository.save(country);
}
```

```
private static void testAddCountry() throws CountryNotFoundException { 1 usage

    LOGGER.info("Start");

    Country country = new Country();
    country.setCode("AU");
    country.setName("Australia");

    countryService.addCountry(country);

    Country result = countryService.findCountryByCode( countryCode: "AU");

    LOGGER.info("Country = {}", result);

    LOGGER.info("End");
}

}
```

```

e : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform
EntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
EnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries ma
tWebServer : Tomcat started on port 8082 (http) with context path '/'
Application : Started OrmLearnApplication in 2.06 seconds (process running for 2.458)
Application : Start
Application : Country = Country{code='AU', name='Australia'}
Application : End

```

2. Spring-data-jpa-handson: HandsOn-1

```

g.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transacti
LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'def
s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be u
aBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, data
s.boot.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path '/'
c.ormlearn.OrmLearnApplication : Started OrmLearnApplication in 2.341 seconds (process running
c.ormlearn.OrmLearnApplication : Start
c.ormlearn.OrmLearnApplication : Containing 'ou'
c.ormlearn.OrmLearnApplication : Containing 'ou' Sorted
c.ormlearn.OrmLearnApplication : Starting With 'Z'
c.ormlearn.OrmLearnApplication : End

```

```

private static void testSearchCountry() { 1 usage

    LOGGER.info("Start");

    LOGGER.info("Containing 'ou'");
    countryService.searchCountries( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Containing 'ou' Sorted");
    countryService.searchCountriesSorted( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Starting With 'Z'");
    countryService.getCountriesStartingWith( alphabet: "Z")
        .forEach(System.out::println);

    LOGGER.info("End");
}
}

```



```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

import com.cognizant.ormlearn.model.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository 2 usages
public interface CountryRepository extends JpaRepository<Country, String> {

    List<Country> findByNameContaining(String text); no usages

    List<Country> findByNameContainingOrderByNameAsc(String text); no usages

    List<Country> findByNameStartingWith(String alphabet); no usages

}

```

```

import java.util.List;
import java.util.Optional;

@Service 3 usages
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional 1 usage
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }

    @Transactional 1 usage
    public void addCountry(Country country) {
        countryRepository.save(country);
    }

    @Transactional no usages
    public List<Country> searchCountries(String text) {
        return countryRepository.findByNameContaining(text);
    }

    @Transactional no usages
    public List<Country> searchCountriesSorted(String text) {
        return countryRepository.findByNameContainingOrderByNameAsc(text);
    }

    @Transactional no usages
    public List<Country> getCountriesStartingWith(String alphabet) {
        return countryRepository.findByNameStartingWith(alphabet);
    }

    @Transactional 2 usages
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {

```

Hands-on 3: Employee, Department, Skill Entity Mapping

```
... : HHH000033: Encountered use of deprecated annotation [interface jakarta.persistence.Identifiable]
... : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable
ctoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
ctories : Hibernate is in classpath; If applicable, HQL parser will be used.
figuration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performe
: Tomcat started on port 8082 (http) with context path '/'
n : Started OrmLearnApplication in 2.135 seconds (process running for 2.623)
n : Start
```

```
package com.cognizant.ormlearn.model;

import jakarta.persistence.*;

@Entity
@Table(name = "department")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "dp_id")
    private int id;

    @Column(name = "dp_name") no usages
    private String name;

    // Generate Getters, Setters and toString()
}
```

```

package com.cognizant.ormlearn.model;

import jakarta.persistence.*;
import java.util.Date;

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name") no usages
    private String name;

    @Column(name = "em_salary") no usages
    private double salary;

    @Column(name = "em_permanent") no usages
    private boolean permanent;

    @Temporal(TemporalType.DATE) no usages
    @Column(name = "em_date_of_birth")
    private Date dateOfBirth;

    |
}

```

```

package com.cognizant.ormlearn.model;

import jakarta.persistence.*;

@Entity
@Table(name = "skill")
public class Skill {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "sk_id")
    private int id;

    @Column(name = "sk_name") no usages
    private String name;
}

```

```

package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Skill;
import org.springframework.data.jpa.repository.JpaRepository;

public interface SkillRepository extends JpaRepository<Skill, Integer> { no usages
}

```

Hands-on 4: Implement Many-to-One Relationship

```

package com.cognizant.ormlearn.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.util.Date;
import java.util.List;

@Getter
@Setter
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    @Column(name = "em_permanent")
    private boolean permanent;

    @Temporal(TemporalType.DATE)
    @Column(name = "em_date_of_birth")
    private Date dateOfBirth;

    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;

    @ManyToMany
    @JoinTable(
        name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id")
    )
    private List<Skill> skillList;
}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service 3 usages
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional 1 usage
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Employee employee) {
        employeeRepository.save(employee);
    }
}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.repository.DepartmentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service 3 usages
public class DepartmentService {

    @Autowired
    private DepartmentRepository departmentRepository;

    @Transactional
    public Department get(int id) {
        return departmentRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Department department) {
        departmentRepository.save(department);
    }
}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Skill;
import com.cognizant.ormlearn.repository.SkillRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service no usages
public class SkillService {

    @Autowired
    private SkillRepository skillRepository;

    @Transactional no usages
    public Skill get(int id) {
        return skillRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Skill skill) {
        skillRepository.save(skill);
    }
}

```

```

private static void testAddCountry() throws CountryNotFoundException { no
    LOGGER.info("Start");

    Country country = new Country();
    country.setCode("AU");
    country.setName("Australia");

    countryService.addCountry(country);

    Country result = countryService.findCountryByCode( countryCode: "AU");

    LOGGER.info("Country = {}", result);

    LOGGER.info("End");
}

private static void testSearchCountry() { no usages

    LOGGER.info("Start");

    LOGGER.info("Containing 'ou'");
    countryService.searchCountries( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Containing 'ou' Sorted");
    countryService.searchCountriesSorted( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Starting With 'Z'");
    countryService.getCountriesStartingWith( alphabet: "Z")
        .forEach(System.out::println);

    LOGGER.info("End");
}

private static void testGetEmployee() { 1 usage

    LOGGER.info("Start");

    Employee employee = employeeService.get(1);

    LOGGER.info("Employee : {}", employee);
    LOGGER.info("Department : {}", employee.getDepartment());

    LOGGER.info("End");
}

```

Hands-on 6: Many-to-Many Relationship (Employee ↔ Skill)

```
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.ManyToMany;
import jakarta.persistence.Table;

import java.util.Date;
import java.util.List;

@Getter
@Setter
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    @Column(name = "em_permanent")
    private boolean permanent;

    @Temporal(TemporalType.DATE)
    @Column(name = "em_date_of_birth")
    private Date dateOfBirth;

    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;

    @ManyToMany
    @JoinTable(
        name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id")
    )
    private Set<Skill> skillList;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id")
    )

    public Set<Skill> getSkillList() {
        return skillList;
    }
}
```

```

public class OmicronApplication {
}

private static void testAddCountry() throws CountryNotFoundException { no us

    LOGGER.info("Start");

    Country country = new Country();
    country.setCode("AU");
    country.setName("Australia");

    countryService.addCountry(country);

    Country result = countryService.findCountryByCode( countryCode: "AU");

    LOGGER.info("Country = {}", result);

    LOGGER.info("End");
}

private static void testSearchCountry() { no usages

    LOGGER.info("Start");

    LOGGER.info("Containing 'ou'");
    countryService.searchCountries( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Containing 'ou' Sorted");
    countryService.searchCountriesSorted( text: "ou")
        .forEach(System.out::println);

    LOGGER.info("Starting With 'Z'");
    countryService.getCountriesStartingWith( alphabet: "Z")
        .forEach(System.out::println);

    LOGGER.info("End");
}

private static void testGetEmployee() { 1 usage

    LOGGER.info("Start");

    Employee employee = employeeService.get(1);

    LOGGER.info("Employee : {}", employee);

    LOGGER.info("Department : {}", employee.getDepartment());

    LOGGER.info("Skills : {}", employee.getSkillList());

    LOGGER.info("End");
}
}

```



```

package com.cognizant.ormlearn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService; // 8 usages
    private static EmployeeService employeeService; // 2 usages
    private static DepartmentService departmentService; // 1 usage
    private static SkillService skillService; // 1 usage

    public static void main(String[] args)
        throws CountryNotFoundException {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        employeeService = context.getBean(EmployeeService.class);
        departmentService = context.getBean(DepartmentService.class);
        skillService = context.getBean(SkillService.class);

        testGetEmployee();
    }

    private static void testGetAllCountries() { // no usages
        LOGGER.info("Start");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.info("Countries: {}", countries);
        LOGGER.info("End");
    }

    private static void testFindCountryByCode() // no usages
        throws CountryNotFoundException {

        LOGGER.info("Start");

        Country country =
            countryService.findCountryByCode("IN");

        LOGGER.info("Country: {}", country);

        LOGGER.info("End");
    }
}

```

```

package com.cognizant.ormlearn.model;

import jakarta.persistence.*;

import java.util.Set;
import jakarta.persistence.ManyToMany;

@Entity
@Table(name = "skill")
public class Skill {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "sk_id")
    private int id;

    @Column(name = "sk_name") // no usages
    private String name;

    @ManyToMany(mappedBy = "skillList") // no usages
    private Set<Employee> employeeList;
}

```

PDF 3 - Hands-on 2: Get All Permanent Employees Using HQL

```
package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> { 2 usages
    @Query("SELECT e FROM Employee e WHERE e.permanent = true") no usages
    List<Employee> getAllPermanentEmployees();
}
```

```
package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service 3 usages
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional 1 usage
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Employee employee) {
        employeeRepository.save(employee);
    }

    @Transactional 1 usage
    public List<Employee> getAllPermanentEmployees() {
        return employeeRepository.getAllPermanentEmployees();
    }
}
```

```

private static void testGetAllPermanentEmployees() { 1 usage

    LOGGER.info("Start");

    List<Employee> employees =
        employeeService.getAllPermanentEmployees();

    LOGGER.info("Permanent Employees:");

    employees.forEach( Employee employee -> {
        LOGGER.info("{} ", employee);
        LOGGER.info("Department : {}", employee.getDepartment());
        LOGGER.info("Skills : {}", employee.getSkillList());
    });

    LOGGER.info("End");
}

```

```

lContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
j.r.query.QueryEnhancerFactories    : Hibernate is in classpath; If applicable, HQL parser will be used.
eConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database
ot.tomcat.TomcatWebServer           : Tomcat started on port 8082 (http) with context path '/'
mlearn.OrmLearnApplication          : Started OrmLearnApplication in 3.257 seconds (process running for 3
mlearn.OrmLearnApplication          : Start
.em_permanent,e1_0.em_salary from employee e1_0 where e1_0.em_permanent=true
mlearn.OrmLearnApplication          : Permanent Employees:
mlearn.OrmLearnApplication          : End

```

```

lContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
j.r.query.QueryEnhancerFactories    : Hibernate is in classpath; If applicable, HQL parser will be used.
eConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database
ot.tomcat.TomcatWebServer           : Tomcat started on port 8082 (http) with context path '/'
mlearn.OrmLearnApplication          : Started OrmLearnApplication in 3.257 seconds (process running for 3
mlearn.OrmLearnApplication          : Start
.em_permanent,e1_0.em_salary from employee e1_0 where e1_0.em_permanent=true
mlearn.OrmLearnApplication          : Permanent Employees:
mlearn.OrmLearnApplication          : End

```

Hands-on 4: Get Average Salary Using HQL

```
te.orm.deprecation : HHH9000033: Encountered use of deprecated annotation [interface jakarta
te.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta
ainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
uery.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
figuration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database c
ncat.TomcatWebServer : Tomcat started on port 8082 (http) with context path '/'
n.OrmLearnApplication : Started OrmLearnApplication in 2.752 seconds (process running for 3.
n.OrmLearnApplication : Start

n.OrmLearnApplication : Average Salary : 0.0
n.OrmLearnApplication : End
```

```
package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;
import java.util.List;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> { 2 usages
    @Query("SELECT e FROM Employee e WHERE e.permanent = true") 1 usage
    List<Employee> getAllPermanentEmployees();

    @Query("SELECT AVG(e.salary) FROM Emp
    Double getAverageSalary(@Param("id")
}

© com.cognizant.ormlearn.repository.EmployeeRepos
@Query("SELECT e FROM Employee e WHERE e.permanent
List<Employee> getAllPermanentEmployees()
orm-learn
```

```

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService; 8 usages
    private static EmployeeService employeeService; 4 usages
    private static DepartmentService departmentService; 1 usage
    private static SkillService skillService; 1 usage

    public static void main(String[] args)
        throws CountryNotFoundException {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        employeeService = context.getBean(EmployeeService.class);
        departmentService = context.getBean(DepartmentService.class);
        skillService = context.getBean(SkillService.class);

        testGetAverageSalary();
    }
}

```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service 3 usages
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional 1 usage
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Employee employee) {
        employeeRepository.save(employee);
    }

    @Transactional 1 usage
    public List<Employee> getAllPermanentEmployees() {
        return employeeRepository.getAllPermanentEmployees();
    }

    @Transactional 1 usage
    public double getAverageSalary(int id) {
        Double avg = employeeRepository.getAverageSalary(id);
        return avg != null ? avg : 0.0;
    }
}

```

Hands-on 5: Get All Employees Using Native Query

```
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-07-07T20:49:38.414+05:30 WARN 22381 --- [orm-learn] [main] org.hibernate.orm.deprecation
2026-07-07T20:49:38.824+05:30 INFO 22381 --- [orm-learn] [main] org.hibernate.orm.core
2026-07-07T20:49:38.908+05:30 INFO 22381 --- [orm-learn] [main] j.LocalContainerEntityManagerF
2026-07-07T20:49:38.947+05:30 INFO 22381 --- [orm-learn] [main] o.s.d.j.r.query.QueryEnhancerF
2026-07-07T20:49:39.421+05:30 WARN 22381 --- [orm-learn] [main] JpaBaseConfiguration$JpaWebCor
2026-07-07T20:49:39.577+05:30 INFO 22381 --- [orm-learn] [main] o.s.boot.tomcat.TomcatWebServe
2026-07-07T20:49:39.580+05:30 INFO 22381 --- [orm-learn] [main] c.c.ormlearn.OrmLearnApplicati
2026-07-07T20:49:39.581+05:30 INFO 22381 --- [orm-learn] [main] c.c.ormlearn.OrmLearnApplicati
Hibernate: SELECT * FROM employee
2026-07-07T20:49:39.625+05:30 INFO 22381 --- [orm-learn] [main] c.c.ormlearn.OrmLearnApplicati
```

```
package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;
import java.util.List;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> { 2 usages
    @Query("SELECT e FROM Employee e WHERE e.permanent = true") 1 usage
    List<Employee> getAllPermanentEmployees();

    @Query("SELECT AVG(e.salary) FROM Employee e WHERE e.department.id = :id") 1 usage
    Double getAverageSalary(@Param("id") int id);

    @Query(value = "SELECT * FROM employee", nativeQuery = true) no usages
    List<Employee> getAllEmployeesNative();
}
```

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service 3 usages
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional 1 usage
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }

    @Transactional no usages
    public void save(Employee employee) {
        employeeRepository.save(employee);
    }

    @Transactional 1 usage
    public List<Employee> getAllPermanentEmployees() {
        return employeeRepository.getAllPermanentEmployees();
    }

    @Transactional 1 usage
    public double getAverageSalary(int id) {
        Double avg = employeeRepository.getAverageSalary(id);
        return avg != null ? avg : 0.0;
    }

    @Transactional no usages
    public List<Employee> getAllEmployeesNative() {
        return employeeRepository.getAllEmployeesNative();
    }
}

```

```

package com.cognizant.ormlearn;

import ...

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService; 8 usages
    private static EmployeeService employeeService; 5 usages
    private static DepartmentService departmentService; 1 usage
    private static SkillService skillService; 1 usage

    public static void main(String[] args)
        throws CountryNotFoundException {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        employeeService = context.getBean(EmployeeService.class);
        departmentService = context.getBean(DepartmentService.class);
        skillService = context.getBean(SkillService.class);

        testGetAllEmployeesNative();
    }
}

```

```
private static void testGetAverageSalary() { no usages

    LOGGER.info("Start");

    double averageSalary = employeeService.getAverageSalary(id: 1);

    LOGGER.info("Average Salary : {}", averageSalary);

    LOGGER.info("End");
}

private static void testGetAllEmployeesNative() { 1 usage

    LOGGER.info("Start");

    List<Employee> employees =
        employeeService.getAllEmployeesNative();

    employees.forEach( Employee employee ->
        LOGGER.info!("{}", employee));

    LOGGER.info("End");
}
```